

情景剧场演绎系统项目详细方案

——基于多智能体内容生产、Unity 数字孪生训练与 ESP32-C3 实时舵机控制
的一体化方案

技术报告

面向系统归档、联调部署与后续迭代使用

二〇二六年四月

目录

摘 要 3

1 项目背景、问题定义与建设目标 4

2 需求分析、系统定位与价值主张 5

3 总体技术架构与完整业务闭环 6

4 软件内容生产与前后端系统设计 7

5 统一剧本协议、数据模型与内容资产组织 10

6 Unity 本地服务、运行时解析与执行调度框架 11

7 数字孪生训练环境与状态空间设计 12

8 基于 PPO 的策略学习方案及示教预热机制 13

9 世界坐标融合、避障裁决与运行安全设计 15

10 实体机器人硬件架构与嵌入式控制实现 15

11 网络通信协议与实时舵机角度直传机制 17

12 Sim-to-Real 迁移、部署监控与闭环优化 17

13 工程实施路线、风险控制与扩展方向 21

14 结论 22

附录 关键接口与示例说明 23

 附录 A 软件到 Unity 的出站说明 23

 附录 B Unity 到机器人控制链路示例 23

 附录 C 推荐观测指标 24

摘要

本报告围绕“情景剧场演绎系统”的整体设计、训练、部署与落地过程，系统整合了三条原本相对独立的技术链路：一是面向创作者和导演端的前后端内容生产系统，负责把主题、灵感、对话与表演意图转化为结构化剧本、角色、场景、分镜、图片与音频等内容资产；二是以 Unity 为核心的仿真训练与执行系统，负责在本地监听服务中接收剧本、完成协议解析、任务调度、数字孪生训练、策略推理与运行时安全裁决；三是以 ESP32-C3 为主控的四舵机机器人狗控制系统，负责通过 WiFi 局域网与 HTTP 协议完成实时角度查询、舵机目标角度设置和实体运动执行。三个子系统在本项目中不再被孤立看待，而是通过统一的结构化剧本、统一的中间表示和统一的数据闭环整合为一套端到端的具身演绎解决方案。

从业务流程上看，系统首先在前端工作台接收用户输入的主题、角色关系、对白片段或表演指令；后端基于多智能体编排和阶段化内容流水线，将输入逐步转化为剧本、角色库、场景库、分镜、提示词、图片与配音，并导出面向 Unity 的统一 JSON 剧本；Unity 本地服务监听指定端口，在收到剧本后完成解析、排队、优先级处理和运行时调度，并结合世界相机坐标、历史状态、脚本标签和环境信息，按固定控制周期持续输出下一时刻四个舵机的目标角度；随后，这些角度通过 HTTP 接口下发给实体机器人，机器人基于 LEDC PWM、FreeRTOS 任务调度和舵机控制逻辑完成动作执行；执行结果与状态信息再回流至 Unity 和上层系统，用于调试、校正与下一轮训练优化。整个系统形成了从高层叙事语义到中层行为编排，再到低层连续舵机控制与实体执行的完整闭环。

在技术实现上，本报告完整覆盖了前端 Nuxt 3 + Vue 3 工作台、后端 Hono + TypeScript 服务、LangChain 多智能体编排、SQLite/Drizzle 数据持久化、多模态素材生成、Unity ML-Agents 数字孪生训练、PPO 强化学习、Behavior Cloning 示教预热、GAIL 辅助模仿、ONNX 模型导出、Sentis 本地推理、HttpListener 本地服务接入、ESP32-C3 HTTP 服务器、实时角度直控、3D 打印结构设计、域随机化、黑箱生成—白名单模板对照优化以及工程可观测性与风险控制等关键内容。报告既力求保持软件侧对结构化资产与工作流的完整描述，也尽量保留仿真训练侧对状态、动作、奖励和部署策略的设计逻辑，同时将机器人侧硬件、嵌入式通信和运动执行的

工程细节纳入统一叙述框架，最终形成一份适合项目答辩、系统归档、后续开发接力和成果申报的完整技术文档。

关键词：情景剧场；多智能体；结构化剧本；Unity；数字孪生；PPO；Behavior Cloning；ONNX；Sentis；ESP32-C3；HTTP；实时舵机角度控制

1 项目背景、问题定义与建设目标

本项目的提出，源于一个明确而实际的需求：传统机器人演示系统往往只能接受低层动作按钮或固定动作序列，虽然能够做出若干预设表演，但很难真正承担“情景剧场演绎”的角色。对于比赛答辩、科普展示和具身智能研究而言，用户希望输入的不是一串机械指令，而是一段主题、一段对白、一条剧情线，甚至只是一个冲突设定或一条高层任务意图；系统需要自动完成中间桥接，让机器人最终表现为“理解了剧本并将其演出来”。因此，本项目的核心问题并不是单个算法的精度，也不是单个硬件模块的速度，而是如何打通从创意输入到实体执行的全链路，使软件生成、仿真训练和实体控制构成统一体系。

从系统工程角度看，问题可以被拆解为五个层次。第一层是内容输入问题：如何接收自然语言主题、角色设定、对白片段、动作意图和导演性修改要求，并将其转化为机器能够处理的结构化输入。第二层是内容生产问题：如何把原始输入不断补全、提炼并沉淀为剧本、角色、场景、分镜、音频和图片等可复用的中间资产。第三层是执行编排问题：如何让 Unity 在收到剧本后不只是显示文本，而是真正承担本地服务、协议解析、动作排队、运行时调度和推理器调用等职责。第四层是低层控制问题：如何把“问候”“转身”“前进”“停顿”“对话配合”等离散语义稳定映射为四个舵机在连续时间上的目标角度流。第五层是现实落地问题：如何将仿真训练得到的策略迁移到真实机器狗，并在真实执行中不断通过数据回流优化性能。

围绕上述问题，本项目建设目标被明确为一条端到端的技术链。其一，构建一个面向用户和创作者的内容生产系统，使非专业编剧也能通过简单输入得到可执行剧本；其二，构建一个基于 Unity 的数字孪生训练与执行系统，使高层剧本可以在仿真中被解释、训练和实时推理；其三，构建一个轻量、可复用、可联网的四舵机机器狗平台，支持通过 WiFi 局域网接收实时角度控制；其四，形成统一协议、统一资产结构和统一调试入口，使三个层次不需要通过大量人工转换就能协同工作；

其五，在工程上保证系统可演示、可迭代、可扩展，而不是仅停留在一次性展示层面。

因此，本报告将“情景剧场演绎系统”定义为一种具身叙事平台：上层接受剧本语义，中层完成行为解释与策略推理，下层输出连续舵机流并驱动真实机器狗执行。只有当内容生产、运行时编排、强化学习策略、网络协议和实体硬件同时被纳入一份统一设计中，系统才真正具备技术闭环意义。

2 需求分析、系统定位与价值主张

需求分析的第一项结论，是系统必须服务于“高层创意输入”，而不是强迫用户直接面对底层控制。因此，系统前端应当允许用户输入一句主题、一个情节冲突、一段散乱对话，甚至只给出角色关系与表现目标。软件侧不能把用户当成程序员，而要把用户视作内容创作者、导演或者讲解者。也正因为如此，系统的软件部分采用了“创作访谈 + 剧本改写 + 结构化提取 + 分镜拆解 + 多模态素材生成”的分阶段流程，而不是一次性对话式生成。

需求分析的第二项结论，是输出不能停留在自然语言层面。对于后续 Unity 解析与机器人执行而言，一段普通的故事文本几乎无法直接消费。系统必须把文本逐步落成剧级策划信息、单集原始草稿、标准剧本、角色资产、场景资产、分镜资产、音频资产和图片资产，并为每一种资产定义可查询、可编辑、可导出的字段。这样做的意义在于，软件侧输出的不再是一段临时性的回答，而是一套可持续维护和复用的内容生产结果。

需求分析的第三项结论，是执行层不应只是“播放预设动作”。对于一个强调具身表达与系统完整性的项目来说，如果机器人只能执行几个硬编码动作，那么上层剧本系统与强化学习系统都会被架空。因而，Unity 侧必须能够根据脚本标签、世界状态和历史记忆，在执行期持续生成下一时刻的舵机角度；机器人侧也必须开放实时角度设置接口，使低层控制成为真正可微调、可回流、可调试的环节。

基于这些需求，本项目的系统定位可以概括为“从创意输入到实体演出的具身表演中枢”。它不是单独的软件工作台，不是单独的仿真训练环境，也不是单独的嵌入式控制板，而是三者有机耦合后的完整平台。它的价值主张则主要体现在三个方面。第一，表演性：输入形式天然适合讲故事、做答辩和做科普展示。第二，可研

究性：系统内部包含多智能体编排、数字孪生训练、强化学习和 Sim-to-Real 迁移，具备学术和工程拓展潜力。第三，可落地性：硬件采用四舵机低自由度方案和 HTTP 控制链路，开发成本低、迭代速度快、现场部署方便。

进一步地，系统在商业化或产品化层面也具备清晰逻辑。对于教育场景，它可以作为具身智能教学平台；对于比赛展示，它可以作为高完成度演示系统；对于研究原型，它可以作为从软件到硬件的验证底座。软件侧的前后端、仿真侧的训练器与执行器、硬件侧的控制器并不互斥，而是共同构成一个可向多个方向延伸的基础平台。

3 总体技术架构与完整业务闭环

总体上，系统采用“内容生产层—执行编排层—实体控制层”的三层主架构，并在其外包裹训练层、监控层和反馈层。内容生产层负责接收用户创意并生产结构化资产；执行编排层负责在 Unity 中监听剧本、解析协议、组织动作节点并驱动策略模型；实体控制层负责把目标角度转化为舵机运动并完成真实动作。训练层和反馈层则贯穿整个系统，用于在仿真阶段学习策略、在现实阶段校正模型，并在全链路中提供可观测性和安全保障。

层次	核心组成	主要职责	输出对象
内容生产层	Nuxt 3 前端、Hono 后端、LangChain 智能体、SQLite/Drizzle、多模态服务	将主题、对白与设定转化为结构化剧本和内容资产	剧本、角色、场景、分镜、图片、音频、JSON
执行编排层	Unity 本地服务、HttpListener、运行时解析器、任务调度器、Sentis 推理器	接收剧本、解析任务、组织动作阶段并持续生成控制量	动作节点、状态向量、舵机目标角度流
实体控制层	ESP32-C3、FreeRTOS、HTTP 服务器、LEDC PWM、四个舵机	接收目标角度并驱动实体机器狗执行	真实舵机状态、姿态变化、执行反馈
训练与反馈层	Unity ML-Agents、PPO、BC、GAIL、日志与监测模块	训练策略、校正策略、回流真实数据并优化系统	模型权重、训练日志、评估指标、修正结果

按照这一架构，业务闭环从用户输入开始。用户在前端工作台输入主题、灵感或对白后，系统先通过创作访谈智能体逐步追问并补齐世界观、角色关系、情绪冲突、表演目标和单集重点，再由剧本改写智能体将原始内容转写为带动作语义的标

准剧本。接着，提取智能体沉淀角色与场景库，音色分配智能体完成角色与声音资源绑定，分镜智能体将剧本拆成具有镜头语义的细粒度节点，提示词与素材生成模块产出角色图、场景图、镜头图与配音文件，最终由后端组装为统一 JSON 剧本并导出到 Unity。

Unity 侧在接收 JSON 后，并不会将其视为普通数据包简单保存，而是立即完成运行时解析。解析器会把剧本中的角色、台词、动作标签、优先级、时长、可打断性和执行约束转化为内部任务节点；调度器则根据当前执行状态决定节点的排队、抢占、切换与结束条件；推理器读取当前机器狗世界坐标、历史动作状态、上一时刻舵机角度、环境风险和动作标签，按固定控制周期输出下一时刻的目标角度向量。

机器人控制系统通过局域网 HTTP 接口接收这些目标角度，并以实时角度直控方式驱动四个舵机完成动作。和传统只支持预设动作编号的方式相比，这种设计使 Unity 可以把高层剧本标签逐步翻译为持续的舵机流，而机器人则不再只是在“播放固定动作”，而是在执行来自上层策略的连续控制命令。执行中产生的状态信息、角度回读或外部世界相机坐标又能够回流到 Unity 与上层系统，用于动态修正与后续训练。

从技术闭环意义上讲，本项目并不是把三个子系统简单“拼起来”，而是明确规定了中间表示和职责边界：软件侧负责生产结构化内容，Unity 侧负责解释和推理，机器人侧负责执行和回读。正是这种分层而非耦合的结构，保证了后续无论更换模型、替换前端界面、改造硬件腿部结构，系统都能在较小改动下继续工作。

4 软件内容生产与前后端系统设计

软件部分的核心定位，是“认知与内容生产中枢”，而不是简单的问答界面。它要解决的关键问题包括：如何让没有编剧经验的用户也能组织出可演绎内容，如何让生成结果不是一次性文本而是可维护的资产，如何让后续执行层可以直接消费这些结果，以及如何通过配置中心、日志系统和阶段化界面形成稳定的工程工作流。因此，软件侧并非采用一句话生成一切的单次调用方式，而是将创意生产拆解为多阶段流水线。

前端采用 Nuxt 3、Vue 3 与 TypeScript 构建，承担项目管理、剧集管理、创作会话、阶段化编辑、结果预览、素材管理和导出控制等职责。与传统聊天式交互不同，

前端更像一个分阶段工作台：用户可以在首页查看项目，在剧项目页管理剧集，在单集页逐步推进“创意草案—标准剧本—角色场景—分镜—图片音频—Unity 导出”等阶段，并在任何节点进行人工修正。这样的设计既适合现场演示，也适合真实开发过程中反复迭代。

后端采用 Hono + TypeScript 实现，负责路由编排、参数校验、数据库读写、任务调度、多模态服务接入以及与 LangChain 智能体层的衔接。后端并不直接把模型结果原样返回，而是通过一组业务域接口将内容拆成 drama、episodes、creative、agent、characters、scenes、storyboards、grid、images、ai-configs、ai-voices、skills 等多个模块，从而保证不同类型的资产能够独立管理、独立修改和独立出站。

在智能体层，系统采用 LangChain 作为编排中枢，并将不同任务拆解为具有明确职责的专职 Agent。creative_interviewer 负责以访谈式交互补齐设定与草案；script_rewriter 负责把原始内容改写成符合执行层消费需求的标准剧本，并尽可能把动作语义提前显性化；extractor 负责从剧本中提取角色、场景及其关系并处理去重问题；voice_assigner 负责基于角色信息与音色库完成声音资源绑定；storyboard_breaker 负责将剧本拆解为镜头级中间表示；grid_prompt_generator 负责为普通图片生成和宫格图 workflow 提供提示词。通过这种多 Agent 分工，系统避免了单模型一次承担过多任务而导致输出混乱的问题。

数据层采用 SQLite + Drizzle ORM。之所以选择这一组合，并不是为了“简单”，而是因为它在竞赛场景和快速迭代场景中具有明显优势：单文件数据库便于迁移与部署，结构化表设计有利于后续查询与对接，Drizzle 提供的类型约束有助于降低字段误用风险。数据库中不仅保存 drama 和 episode 级别的文本内容，还维护 characters、scenes、storyboards、image_generations 等关键对象，使每一步生成结果都能沉淀为独立资产。

软件侧还集成了图片生成和 TTS 音频服务。图片服务会根据当前激活的图像模型配置，读取角色、场景、镜头或宫格图提示词，调用对应供应商接口生成图片，并将结果写入本地静态目录；TTS 服务则根据角色音色与台词文本生成试听音频和镜头级配音。系统在模型管理层支持多供应商并存，例如文心链路、图片生成链路和音频服务链路可以分别选择最适合当前演示或答辩的配置，而剧集级别的配置锁定机制又能保证同一集内容在风格上保持一致。

围绕工程可用性，软件系统还设计了配置中心、技能包机制和任务日志系统。配置中心不是静态表单，而是运行时控制台，允许测试文本、图片、音频服务的有效性，设置默认模型并切换优先级；Skill Pack 机制允许不同 Agent 挂载各自的技能文档与默认行为包，从而在线调整其表现方式；任务日志会对图片生成、TTS 请求、配置探测、接口调用和轮询过程进行完整记录，并对 API Key、Token、URL 等敏感信息做脱敏处理，为联调和问题定位提供依据。

智能体/模块	职责	典型输入	典型输出
creative_interviewer	访谈式补齐创意设定和单集草案	主题、对话、历史会话、当前草案	结构化草案、下一轮追问焦点、提交条件
script_rewriter	将草案或原始文本改写为标准剧本	原始内容、整剧设定、单集目标	格式化剧本、动作语义标签
extractor	提取角色与场景并维护世界观一致性	标准剧本、已有角色场景库	角色表、场景表、关联关系
voice_assigner	绑定音色与角色	角色信息、音色资源池、音频配置	角色-音色映射、试听任务
storyboard_breaker	将剧本拆解为镜头级中间表示	剧本、角色、场景	镜头编号、景别、运镜、对白、动作、时长
grid_prompt_generator	面向普通生图和宫格图生成提示词	镜头语义、风格要求、参考图	prompt、cell prompt、批量图生成输入

从关键业务流程来看，软件侧首先完成“从创意到草案”的访谈式推进，再进入“从草案到剧本”的改写流程；接着将剧本转化为角色场景资产，继续向下完成音色分配、分镜拆解、图片生成、宫格图生成和 TTS 生成，最后统一组装为面向 Unity 的 JSON。整个过程中，阶段化 UI 允许用户在任意阶段中断、回改和重生成，从而形成 AI 生成与人工审核互相配合的人机协同闭环。

软件系统的一项关键设计，是把分镜层明确为“真正的中间表示”。如果系统只保留剧本和最终角度，那中间很多信息都会丢失，难以支持图片、配音、动作和节奏的统一协调；而有了分镜层，每个镜头都可以拥有景别、角度、运镜、对白、动作、氛围、时长、图像提示词、音乐提示词和音效提示词等字段，既有利于多模态素材生成，也有利于 Unity 把高层内容映射成具体动作阶段。

软件侧的另一项设计亮点，是将“文心优先但不被单一供应商锁死”的策略落到配置系统中。文本、图片和音频服务分别以配置中心统一管理，支持多服务商混搭，同时通过剧集级配置锁定保证单次演示过程中的风格一致与资源稳定。对比赛和现场答辩而言，这种设计兼顾了技术路线表述的完整性与实际部署时的灵活性。

5 统一剧本协议、数据模型与内容资产组织

为了把软件侧内容生产结果稳定送入 Unity 与实体机器人，系统需要一个清晰、统一、可扩展的中间协议。这里的“统一剧本协议”并不是单纯的字符串格式，而是一组围绕项目、角色、场景、分镜、时间线与执行约束组织起来的数据对象。其设计目标在于：第一，使上层生成结果对下层解析器来说易于消费；第二，使协议既能表达叙事信息，也能表达动作和时序信息；第三，使后续扩展更多角色、更多场景和更多执行器成为可能。

在数据模型设计上，系统将内容拆成剧级与集级两类对象。剧级对象负责维护整体主题、世界观、角色圣经和整剧大纲；集级对象负责维护当前单集的原始草稿、标准剧本、单集目标和当前流水线状态。再往下，`characters` 表负责角色名、外观、性格、音色和角色图；`scenes` 表负责地点、时间段、场景描述和场景图；`storyboards` 表负责镜头级别的景别、运镜、对白、动作、氛围与时长；`image_generations` 表则记录普通生图、宫格图、切分回填和参考图等任务状态。这种表结构设计使内容生产结果具备长期可维护性。

统一 JSON 的出站结构一般包含 `project`、`episode`、`characters`、`scenes`、`storyboards` 和 `timeline` 等部分。`characters` 为 Unity 提供角色映射与资源引用，`scenes` 提供地点和背景语义，`storyboards` 则承担镜头级任务输入，`timeline` 指明镜头的衔接顺序和时间推进关系。若系统已生成图片与音频文件，则这些资源路径也将被写入 JSON，以便 Unity 在执行期同步展示或播放。

从对接效果上看，统一协议最大的价值，是避免上层系统每次都向 Unity 发送大段自然语言文本，再由 Unity 自己重新解析。这种做法会造成不稳定和重复工作。通过结构化出站，软件系统已经完成了大部分语义拆解工作，Unity 则只需专注于运行时解析、阶段调度与动作控制，从而显著降低联调难度。

```
{
  "project": {"title": "微境剧场演示", "theme": "医疗问诊情境"},
  "characters": [
    {"id": "doctor", "name": "医生", "voice": "warm_male"},
    {"id": "dog", "name": "机器狗", "role": "表演主体"}
  ],
  "scenes": [
    {"id": "clinic_room", "place": "诊室", "time": "白天"}
  ],
  "storyboards": [
```

```
{
  "id": "shot_01", "action": "walk_forward", "duration": 2.0,
  "dialogue": "请跟我来"},
  {
    "id": "shot_02", "action": "greet", "duration": 1.2, "dialogue": "
    你好"}
  ],
  "timeline": ["shot_01", "shot_02"]
}
```

虽然实际协议会比示例更复杂，包含优先级、约束条件、媒体资源、回调字段和运行时标记等内容，但其核心思想是一致的：上层以结构化剧本输出叙事意图，下层以结构化剧本输入执行任务，从而使三大子系统围绕同一种中间表示协同。

6 Unity 本地服务、运行时解析与执行调度框架

在本项目中，Unity 并不是传统意义上只负责画面渲染的引擎，而是运行时中枢。它在部署阶段承担本地监听服务、剧本协议接入、运行时解析、队列管理、动作调度、策略模型推理和安全裁决等多项职责，是连接软件内容系统和实体机器人之间的关键桥梁。这样设计的原因在于，剧本到动作之间存在较复杂的时序转换和环境感知需求，如果把这些逻辑完全放在前端或嵌入式侧都不合适，而 Unity 恰好具备场景管理、状态更新和模型运行能力。

Unity 侧通过本地 HTTP 服务监听约定端口，用于接收来自软件系统的统一 JSON 剧本。服务接入层会首先完成请求合法性校验、协议反序列化、资源路径确认和基本字段检查，然后把结果交给运行时解析器。解析器会把角色、台词、镜头、动作标签、时长、优先级、可打断性和执行约束装配为内部任务结构，为后续调度器和推理器提供标准输入。

调度器是 Unity 运行时中的核心逻辑之一。其职责不只是顺序播放镜头，而是根据当前执行状态决定哪一个动作节点应该开始、继续、结束、打断或切换。例如，当一个“前进”动作正在执行时，如果新的高优先级镜头要求立即“转向并停顿”，调度器需要按照预设规则完成节点抢占与状态过渡。为了让剧场演绎更自然，调度器还需考虑对白节奏、镜头时长和动作连续性，而不是简单依赖固定时间片。

在具体执行时，Unity 会把来自调度器的当前动作标签、剩余时长、当前阶段、世界状态和历史动作记忆共同送入推理模块。推理模块不会一次性输出整段轨迹，而是在每一个控制周期内，根据当前状态推算下一时刻四个舵机的目标角度。这一

“按周期自回归输出舵机角度流”的方式是系统的关键特征，它使机器狗在执行期能够持续根据环境和阶段推进进行细微调整，而不是死板地复现某条离线轨迹。

模型部署上，Unity 采用 ONNX 作为中间格式导出策略模型，并在运行时利用 Sentis 或相关推理组件执行本地神经网络推理。这样做一方面减少了对外部服务的依赖，使演示过程不必实时联网调用云端模型；另一方面也使 Unity 侧能够把输入状态、推理输出和场景变化纳入同一进程中处理，降低系统时延和接口复杂度。

为了便于联调和维护，Unity 服务侧还会暴露状态查询、当前任务查看、位姿更新或调试输入等接口，使得外部工具可以了解当前任务节点、当前世界坐标、当前输出角度以及是否触发了避障或安全裁决机制。这样，Unity 不仅是执行者，也成为系统级调试的中心观测点。

运行时模块	作用	关键输入	关键输出
接入层	监听端口、接收 JSON 剧本、完成协议校验	HTTP 请求、JSON 剧本	解析后的任务包
解析器	将剧本映射为内部任务节点和镜头队列	角色、场景、storyboard、约束信息	动作节点、时间线、调度参数
调度器	控制节点排队、切换、抢占和结束	当前节点、优先级、时长、执行状态	当前动作标签、阶段、剩余时长
推理器	根据状态持续输出下一时刻舵机目标角度	状态向量、动作标签、历史记忆	四个舵机的目标角度
安全裁决器	检查碰撞、越界、墙体风险和异常状态	世界坐标、障碍信息、动作输出	限幅结果、停止指令、替代动作

7 数字孪生训练环境与状态空间设计

要让 Unity 运行时具备根据剧本实时生成动作的能力，仅靠手工规则或固定插值很难兼顾流畅性、稳定性与可泛化性，因此本项目在 Unity 中构建了数字孪生训练环境，并将强化学习作为策略优化主线。所谓数字孪生，并不仅指搭建一个外观相似的 3D 模型，而是要求训练环境在物理属性、关节约束、动作时间尺度和观测方式上尽量贴近真实机器狗，使得训练出来的策略能够具备现实落地可能。

训练环境中的智能体主体是四舵机低自由度机器狗。每一条腿在当前原型中仅配置一个自由度，因此系统需要在低自由度条件下完成前进、后退、转向、问候、停顿、姿态变化等多类表演动作。这种条件既带来了约束，也形成了研究意义：在动作维度有限的情况下，如何通过标签条件、自回归输出和强化学习奖励设计获得看起来连贯且具表达性的表演行为，是整个系统训练层的核心难点。

状态空间设计需要同时服务于“会动”和“会按剧本动”两个目标。为此，系统把状态向量设计为多源信息的组合：既包含机器狗当前世界坐标、朝向、速度、角速度和机体高度等运动学信息，也包含四个舵机当前角度、上一时刻目标角度、历史动作窗口、当前动作标签、动作剩余时长、当前镜头阶段以及来自世界相机或场景感知模块的障碍距离和风险标记。通过把脚本条件和环境状态统一纳入状态空间，策略模型才能在运行时真正实现“按剧情且看环境”地输出动作。

动作空间设计与机器人控制接口保持一致。模型的直接输出为四个舵机在下一控制周期的目标角度，或在某些实现中为相对于当前角度的增量。选择连续动作而非离散动作库，有利于保证平滑性与可调性，也便于与实体机器人的实时角度设置端点自然对接。与此同时，系统会为动作输出设置限幅与平滑约束，以避免策略在训练早期输出过大幅度的抖动。

为了增强训练环境的实用性，数字孪生侧还引入了示教动画、模板动作回放、场景扰动和多环境并行训练等机制。示教动画用于提供起步阶段的专家行为轨迹，模板动作回放则为机器人提供一套稳定可复现的参考动作，场景扰动用于提升策略对不确定性的适应能力，多环境并行训练则加快 PPO 收敛速度。整体设计并不是追求极度复杂的机器人动力学，而是在有限开发周期内构建一个足以支持策略学习与现实迁移的有效训练底座。

状态类别	典型字段	设计目的
机体状态	世界坐标、朝向、线速度、角速度、机体高度	反映当前位姿与运动趋势
舵机状态	FL/FR/BL/BR 当前角度、上一时刻目标角度	保证控制连续性并降低抖动
脚本条件	当前动作标签、镜头阶段、剩余时长、角色语义	让策略输出符合剧情要求
历史记录	最近若干步动作与状态窗口	支持自回归控制与阶段延续
环境安全	障碍距离、墙体方向、风险标记	避免策略把机器人推向危险区域

8 基于 PPO 的策略学习方案及示教预热机制

在训练方法上，系统选择 PPO 作为强化优化主线，同时配合 Behavior Cloning 示教预热，并在需要时引入 GAIL 作为辅助模仿手段。这一组合的原因非常直接：如果仅依赖监督学习去拟合示教动作，模型虽然可以“像”专家一样运动，但难以在环境扰动、动作切换和安全约束下主动做出更优调整；如果一开始完全从零开始强

化学习，又会面临探索效率低、动作极不稳定和训练样本浪费的问题。因此，本项目采用“示教初始化 + PPO 强化优化 + 真实回流校正”的分阶段闭环，是在工程可行性与策略性能之间取得平衡的一种合理路径。

Behavior Cloning 阶段的核心任务，是通过 Unity 数字孪生环境中的动作动画示教和模板动作回放构建专家数据集。具体来说，系统录制前进、后退、转向、问候、停顿等典型动作在不同阶段下的舵机目标角度与状态序列，并把这些样本整理成可供模仿学习使用的示教数据。BC 阶段的目标不是让策略完全依赖示教，而是让模型先掌握“站稳”“不乱抖”“会基本动作”的初始能力，从而避免 PPO 在训练初期因动作过于随机而难以获得有效回报。

在进入 PPO 阶段后，策略的目标从“模仿已有动作”转变为“在奖励约束下主动做得更好”。奖励函数通常由多项组成：一类是任务完成奖励，用于鼓励策略按照当前动作标签或脚本阶段完成预期动作；一类是运动质量奖励，用于鼓励平滑、稳定、协调的动作输出；一类是进度奖励，用于鼓励在前进、转向等任务中获得正确方向的位移或姿态变化；还有一类是惩罚项，用于抑制碰撞、越界、过大能耗、过快抖动和违反安全约束的行为。通过多目标奖励组合，PPO 能够将“看起来像”和“在执行中更稳”统一起来。

为了增强从示教到强化的衔接，系统还预留了 GAIL 辅助模仿路径。当某些动作标签难以通过简单 BC + PPO 稳定学习时，引入对抗式模仿可帮助策略更好地区分“像专家”和“不像专家”的行为分布。不过在当前工程阶段，GAIL 更多被视为补充选项，而不是每个动作都强制使用的标准流程。项目整体仍以 BC 打底、PPO 主训为核心。

训练阶段还会使用并行环境和配置驱动方式管理实验。每一次训练都会记录行为名称、场景参数、奖励系数、动作频率、环境随机化范围和模型检查点，确保不同实验结果可对照、可回溯。这样一来，策略训练就不再只是“试一试能不能跑起来”，而是形成了相对规范的实验记录体系。

训练阶段	主要目标	核心输入	产出
示教数据构建	录制专家轨迹并建立模板动作数据集	示教动画、模板动作、状态-动作序列	示教样本库
Behavior Cloning 预热	让策略先学会基本稳定动作	示教样本库	具备初步控制能力的初始策略
PPO 强化优化	在奖励约束下提高稳定	初始策略、并行环境、	强化优化后的策略模型

	性与任务完成度	奖励函数	
GAIL 辅助模仿	在难动作上增强模仿效果与风格一致性	专家样本、策略轨迹	更接近专家分布的策略
真实回流校正	缩小现实鸿沟并修正部署偏差	真机执行数据、日志、位姿回流	可部署版本模型与修正参数

9 世界坐标融合、避障裁决与运行安全设计

剧本驱动系统要想真正落地，运行期安全绝不能被忽略。因为强化学习策略即使在仿真中表现良好，部署到真实环境时仍可能因为地面摩擦、舵机延迟、障碍物分布和定位误差而输出危险动作。为此，本项目在 Unity 运行时中引入了世界相机坐标融合和避障裁决机制，使模型输出不再被无条件下发给机器人，而是在执行前经过安全过滤。

世界坐标融合的核心目标，是将真实机器狗或仿真狗在场景中的位置、朝向与外部感知信息统一纳入运行时状态。Unity 可通过世界相机、视觉标记或其他定位方式获得机器狗的世界坐标，并在每个控制周期内将其写入状态向量。这样，策略不仅知道自己“该做什么动作”，也知道自己“现在处于什么位置、面朝哪个方向、是否接近边界或障碍物”。

安全裁决器工作在策略输出之后、控制命令下发之前。它会检查当前角度输出是否可能导致越界、撞墙、剧烈抖动或危险姿态。如果检测到风险，系统可以采用多种处理策略：对角度做限幅裁剪、降低控制幅度、强制插入停顿动作、切换到白名单保底动作、直接下发停止指令，或者请求上层调度器重新规划当前节点。这种“策略输出 + 外部安全盾”的结构，在工程上比单纯依赖模型自我约束更稳妥。

对于剧场演绎场景而言，安全机制还承担了节奏保护功能。例如在对白阶段，系统不希望机器狗因一次错误的高幅度动作打断表演流程；在狭小舞台范围内，系统也不希望一个前进动作把机器人直接送出场景。因此，安全设计并不是附属补丁，而是和调度器、推理器并列的关键模块。

10 实体机器人硬件架构与嵌入式控制实现

实体机器人部分采用四舵机低自由度机器狗方案，以 ESP32-C3 作为主控芯片，以 3D 打印结构件作为机械基础，并通过局域网 HTTP 协议对外提供实时控制接口。选择这一方案的出发点并不是追求复杂机械性能，而是希望在较低成本、较短周期

和较高可维护性的条件下，构建一套足以承载剧场演示和策略验证的实体平台。四舵机低自由度结构虽然动作空间有限，但它恰好与本项目关注的“高层语义—中层编排—低层连续控制”链路相匹配：通过有限自由度做出有辨识度的动作，更能体现控制系统与内容系统的协同价值。

在机械实现上，机器人大量采用 3D 打印技术。这一选择带来了多重优势。首先，3D 打印降低了从设计图纸到实物原型的转换门槛，使结构件能够快速迭代。其次，针对腿部连接件、舵机安装槽和机身支撑件等关键部位，可以在设计阶段就根据舵机尺寸和布线需求进行定制。再次，当仿真训练和现实执行发现结构性问题时，团队能够较低成本地修改模型并重新打印，而不需要等待复杂机械加工。

硬件架构上，ESP32-C3 既负责 WiFi 驱动和 HTTP 服务器，也负责舵机控制、指令解析和任务调度。系统采用 LEDC PWM 输出作为舵机控制基础，通过把角度目标映射为对应占空比，驱动前左、前右、后左、后右四个舵机完成姿态变化。为了保证执行的实时性和结构清晰度，嵌入式侧基于 FreeRTOS 将网络接收、指令处理、角度更新和动作执行拆分为不同任务或任务阶段，并借助队列或共享状态变量在模块之间传递目标值。

除了“会动”之外，机器人侧还强调“可查、可调、可校准”。系统不仅支持按动作名称调用预设动作，也支持实时查询当前四个舵机角度，并直接设置四个舵机的目标角度。这样，软件系统和 Unity 不必每次都退回到低灵活性的动作编号模式，而可以在需要时发送精确的目标姿态。对于强化学习部署而言，这一点尤其重要，因为模型输出天然就是连续数值，而不是离散动作标签。

在步态与动作组织方面，嵌入式侧保留了动作函数表与基础步态逻辑，用于在无模型、网络异常或保底演示时提供白名单能力。例如前进、转向、问候、抬腿、重置等动作可以由嵌入式动作表直接触发。这样的设计一方面让系统具备调试和容错基础，另一方面也为后续的“黑箱生成—白名单模板对照优化”提供了现实参考。

硬件层次	关键组件	说明
主控层	ESP32-C3	负责 WiFi 通信、HTTP 服务、指令解析与 PWM 输出
执行层	FL/FR/BL/BR 四个舵机	承担实际姿态变化与运动输出
机械层	3D 打印机身、腿部连接件、舵机安装结构	支持快速成型和快速迭代
供电与连接层	电源模块、连线、接口与	保证舵机供电稳定和结构可靠

	固定结构	
软件支撑层	FreeRTOS、LEDC、HTTP 路由	为实时控制和网络接口提供基础能力

11 网络通信协议与实时舵机角度直传机制

机器人控制系统在通信层不再采用无状态的 HTTP 请求—响应方式作为主控制链路，而是建立一条持续保持的双向数据通道，作为 Unity 仿真端与嵌入式机器人之间的核心实时通信机制。之所以采用数据通道而不是 HTTP，本质原因在于本系统并不是一个低频的人机交互系统，而是一个需要持续下发控制量、持续接收状态反馈的实时运动控制系统。在这种场景下，HTTP 虽然具备良好的通用性与可调试性，但其短连接式请求组织方式、重复报文头开销以及天然偏向“离散事务调用”的通信模型，并不适合高频连续角度控制。相比之下，建立专用数据通道后，系统可以在一次建链成功后保持长连接状态，使上层控制器能够以固定频率持续发送目标角度，同时嵌入式侧持续回传当前姿态、执行状态和异常标志，从而形成真正意义上的闭环数据流。

从总体结构上看，本项目将通信链路设计为“上层调度端—中间数据通道—下位机执行端”的三层模式。其中，上层调度端主要由 Unity 本地服务承担，负责根据剧本节点、策略网络输出以及当前运行状态生成下一时刻的四舵机目标角度；中间层由局域网内的持久化 Socket 数据通道构成，专门用于传输控制帧和状态帧；下位机执行端由 ESP32 所在的嵌入式控制程序构成，负责解析控制帧、更新目标角度、驱动 PWM 输出，并把执行后的状态重新发送到通道中。这样设计的优点在于，上层输出的是连续控制量，中层传输的也是连续控制量，下层接收后无需再经过“动作名称到角度模板”的二次映射，从而避免了离散化带来的精度损失和响应迟滞。

在具体实现中，系统采用基于 TCP 的长连接数据通道作为主链路。选择 TCP 而不是 UDP，是因为机器人演绎场景更强调控制指令的有序到达、完整到达和状态可确认性。虽然 UDP 在理论上延迟更低，但其无连接、无重传、无顺序保障的特性更适合传感器广播或高容错流媒体，而本系统中四舵机角度控制直接影响实体机器人当前姿态，一旦丢包或乱序，容易引起机器人动作突变。因此，项目在工程实践中优先保证“稳定可靠”而非“极限低延迟”，使用 TCP 建立一条持久保持的控制链路。Unity 端作为主动连接方，在程序启动后自动连接机器人控制器的监听端口；

ESP32 端作为服务端，初始化 WiFi 成功后启动 Socket 监听任务，一旦握手成功，即进入控制帧接收与状态帧发送的循环。

为了保证数据通道中的消息边界清晰，系统没有直接把字符串拼接后裸发，而是设计了一套轻量级应用层帧格式。每一帧数据由帧头、负载区和校验区三部分组成。帧头中至少包含消息类型、序号、时间戳和负载长度等字段；负载区承载具体业务数据，例如四个舵机角度、当前模式编号、电池电压、连接状态或错误码；校验区则用于快速检测传输过程中的帧破坏。通过这种“自定义帧协议 + 持久连接”的方式，系统既保留了长连接实时性，又避免了传统文本协议在高频控制中容易出现粘包、拆包和边界不清问题。对于嵌入式侧而言，接收到字节流后，先在接收缓冲区中查找合法帧头，再根据长度字段截取完整消息进行解析；如果长度不足，则继续缓存等待下一批字节到达；如果校验失败，则丢弃该帧并记录异常计数。这一机制使数据通道在复杂网络环境下仍具有较高鲁棒性。

在消息语义上，系统并未将所有信息混杂为同一种结构，而是划分为若干明确的数据帧类型。首先是控制帧，用于由 Unity 向机器人发送四个舵机的目标角度、目标过渡时间以及本帧序号等信息；其次是状态帧，由机器人定期回传当前实际角度、目标角度、当前控制模式、最近一次错误状态和链路质量等；第三类是心跳帧，用于在上层短时间未下发新控制量时维持链路活性，并让下位机判断控制端是否仍在线；第四类是事件帧，主要用于发送如“动作完成”“超限保护触发”“姿态异常”“重连成功”等低频但重要的系统事件。通过这种分层消息类型设计，控制流、反馈流和系统管理流被清晰分开，既提高了协议可维护性，也为后续扩展更多传感器数据或执行器节点提供了空间。

从控制周期设计来看，数据通道不是按“用户点击一下按钮就发送一次命令”的思路组织的，而是遵循固定频率更新。Unity 端会以一定的控制频率周期性产生目标角度，例如每 20 ms、30 ms 或 50 ms 输出一帧控制数据。每次发送的不是抽象动作标签，而是当前时刻四个舵机应该逼近的具体目标值。嵌入式侧收到后，并不会简单地瞬间把舵机强制拉到该角度，而是先将目标角度写入内部共享状态区，再由底层控制循环读取目标值，结合上一时刻的当前位置进行渐进式更新。这样做的目的，是避免由于上层输出变化较大而引发舵机机械冲击。换言之，数据通道传的是

“当前控制目标”，底层 PWM 驱动层执行的是“有限速、有限幅、可保护的过渡控制”。这使得系统在保持实时性的同时，也兼顾了硬件安全性。

嵌入式端的软件结构围绕这一数据通道进行了重新组织。在 ESP32 侧，通常至少划分为三个核心任务：网络接收任务、控制执行任务和状态回传任务。网络接收任务专门负责 Socket 收包、协议解析和目标值更新，不直接操作舵机；控制执行任务以固定周期运行，读取最近一次有效目标角度，对角度进行限幅、插值和平滑处理后映射为 PWM 占空比，并最终驱动四个舵机；状态回传任务则负责周期性采集当前角度状态、连接状态和错误标志，封装成状态帧回发给 Unity。三者之间通过共享内存区或消息队列进行通信，而不是相互阻塞调用。这种任务分离方式能够保证：即便网络瞬时抖动，底层控制循环也不会被拖慢；即便舵机执行周期固定，状态上报也不会堵塞控制收包线程，从而维持整个系统的实时性和稳定性。

为了让上层能够确认控制是否真正生效，系统在数据通道协议中设计了序号与确认机制。每一帧控制消息都带有递增序号，嵌入式端在成功解析并接受该帧后，会在随后的状态帧中带回最近一次已接收的控制序号以及当前执行状态。这样，Unity 就能够区分“命令是否发出”“命令是否被收到”“命令是否正在执行”以及“命令是否基本到位”这几个不同层次。对于强化学习部署和现场演示而言，这种显式确认非常重要，因为它避免了上层“以为机器人已经动了，实际上指令还停留在网络层”的状态不一致问题。此外，当发现序号持续不更新、心跳超时或状态帧中的错误位被置位时，上层还可以立即进入保护逻辑，例如暂停剧情推进、切换为默认安全姿态或触发重连。

在安全性设计上，虽然主链路改为了实时数据通道，但系统并没有放弃必要的保护机制。嵌入式侧会对每一个舵机角度输入进行合法性检查，具体包括数值格式检查、上下限范围约束、相邻两帧变化幅度限制以及异常值过滤等。若上层发来的某一角度明显越界，系统不会直接执行，而是将其裁剪到安全范围内，并把异常标志写入下一帧状态反馈中。对于长时间未收到控制帧的情况，嵌入式端还会启动超时失联保护：一旦超过设定时间阈值仍未接收到新的有效控制或心跳数据，下位机会自动停止当前动态演绎过程，将四舵机缓慢回归到预设中立姿态，以避免机器人因链路中断而停留在危险位置。由此可见，数据通道虽然追求实时性，但其工程实现并不是“裸奔式直传”，而是在每一层都增加了保护与约束。

除了主控制数据通道外，系统仍然保留了一个较轻量的辅助接口层，但其作用已经从“主控制路径”降级为“调试与监控路径”。例如，在开发阶段可以通过简单的调试页面或 Python 脚本连接到同一数据通道，主动发送测试控制帧，验证某一组角度是否合理；也可以通过串口或日志接口观察当前帧解析情况、丢包计数和执行状态。这样一来，系统在正式运行时依赖的是高效的持久通道，在测试和展示时又依然保留了足够的可观测性，不会因为全面切换到底层通信方式而失去调试便利。

从整体控制链路角度看，数据通道方案的最大价值在于它真正打通了“剧本标签—策略推理—连续控制—实体执行”之间的数值级映射关系。上层 Unity 根据剧本和策略模型输出连续角度，中间通道原样传输连续角度，下位机按照连续角度进行约束执行，并把结果以状态帧形式实时反馈回来。这意味着系统不再依赖“动作编号表”“预设动作库索引”作为主要桥梁，而是建立了一条更加直接、更加灵活、也更适合智能控制系统的运行路径。对于情景剧场演绎系统而言，这种数据通道不仅仅是一种通信技术替换，更是整个系统从“离散命令式机器人”向“连续控制式具身体”演进的重要基础。

为便于在技术报告中表达该通道设计，通信消息可概括为如下几类：

数据帧类型	方向	作用	典型负载
ControlFrame	Unity → Robot	下发四舵机目标角度与时序信息	fl, fr, bl, br, seq, duration
StateFrame	Robot → Unity	回传当前角度、目标状态、最近确认序号	cur_fl, cur_fr, cur_bl, cur_br, ack_seq, mode
HeartbeatFrame	双向	保活、检测链路是否在线	timestamp, node_id
EventFrame	Robot → Unity	上报异常、完成、超限、重连等事件	event_code, detail
SyncFrame	Unity ↔ Robot	初始握手、参数同步、模式切换	protocol_ver, robot_id, control_mode

12 Sim-to-Real 迁移、部署监控与闭环优化

仿真策略迁移到现实环境，是本项目真正走向“完整系统”必须跨过的一道门槛。无论数字孪生环境构建得多逼真，真实机器人仍会在舵机响应延迟、供电波动、地面摩擦、结构松动、重心偏差和定位噪声等方面与仿真存在差异。因此，系统在设计之初就把 Sim-to-Real 迁移纳入主链路，而不是等到最后临时补救。

迁移策略首先依赖域随机化。也就是说，在 Unity 训练阶段，系统不会把所有物理参数固定死，而是对质量、摩擦系数、阻尼、控制延迟、动作噪声和初始位置等参数做一定范围内的扰动，让模型在训练期就学会面对“不完全一致”的环境。这样的策略虽然不能完全消除现实鸿沟，但能够显著提升部署时的鲁棒性。

其次，系统强调真实数据回流。实体机器狗执行脚本期间，Unity 和嵌入式侧都会记录目标角度、回读角度、执行阶段、位姿轨迹、异常触发和安全裁决结果。通过对这些真实执行数据进行分析，团队可以识别哪些动作在现实中最容易失真，哪些奖励项需要重权重，哪些动作更适合作为白名单保底模板，从而在后续训练中引入残差修正或动作模板对照。

所谓“黑箱生成—白名单模板对照优化”，指的是在部署阶段既保留强化学习策略作为黑箱生成器，又保留一组经过人工校验或规则编写的白名单动作模板。当模型输出与安全约束冲突时，系统可以临时退回白名单动作；当模型输出与模板高度接近时，又可用模板轨迹作为现实参考对模型进行偏差分析。这样做不是否定学习策略，而是在工程上增加一层保底机制和分析抓手。

监控与可观测性是迁移闭环的另一关键。软件侧记录内容生成过程，Unity 侧记录协议接入、动作调度、推理状态和安全事件，嵌入式侧记录 HTTP 请求、角度更新、队列状态和执行结果。通过把三条日志链统一到调试流程中，项目可以较快定位是上层剧本问题、运行时解析问题、模型推理问题还是硬件执行问题，从而避免“出错了但不知道在哪一层”的情况。

13 工程实施路线、风险控制与扩展方向

从实施路线看，项目可以自然分为若干阶段。第一阶段是环境与基础设施搭建，包括软件前后端骨架、Unity 工程、ESP32-C3 开发环境与 3D 打印原型准备。第二

阶段是核心数据模型与内容流水线落地，完成项目、剧集、角色、场景和分镜对象建模。第三阶段是智能体链路和多模态素材生成落地，打通从创意到 JSON 出站的完整流程。第四阶段是 Unity 本地服务、数字孪生环境和初始示教训练落地。第五阶段是实体机器人硬件装配、HTTP 接口与实时角度控制落地。第六阶段是系统联调、仿真到现实迁移和现场演示优化。这样的阶段划分既贴合开发实际，也有利于在多人协作时控制接口边界。

风险控制方面，系统主要面临四类问题。第一类是内容风险，即剧本生成质量不足、角色设定不稳定、分镜粒度不合适，导致下游执行信息不明确。对此，需要通过访谈式前置策划、人工确认节点和结构化字段校验降低风险。第二类是训练风险，即策略在仿真中学会了某些不稳定行为或过拟合了特定场景。对此，需要通过 BC 预热、多环境并行、奖励调整和域随机化提升稳定性。第三类是部署风险，即现实环境下的响应延迟和机械误差导致动作失真。对此，需要通过白名单模板、角度限幅、安全裁决和回读确认机制降低损害。第四类是联调风险，即不同模块接口不统一、字段含义不一致或日志不可追溯。对此，需要通过统一 JSON 协议、接口文档和三层日志体系保证协作可控。

从扩展方向看，软件侧可以继续增强整剧级策划、角色关系图谱、多角色协同分镜和自动配乐等能力；Unity 侧可以继续引入更复杂的环境感知、更细粒度的状态编码和更高质量的动作风格学习；机器人侧则可以向多自由度腿部结构、更强的传感器输入和更完善的状态回传发展。由于本项目已经在架构上完成三层解耦，后续扩展不必推翻现有系统，只需围绕中间表示和接口进行增量升级。

在工程落地计划上，当前系统最重要的不是盲目增加功能，而是持续提高“可解释、可部署、可复现实验”的程度。因此，未来推进的优先级应当依次放在：统一数据协议、完善训练与部署日志、增加现实回流样本、提高白名单动作质量、优化现场演示稳定性、扩展更多表演动作标签。这样可以保证系统既有研究深度，也有实际演示效果。

14 结论

本报告将原本分别聚焦于软件内容生产、Unity 仿真强化学习和嵌入式机器人控制的三份技术文档，整合为一套完整的“情景剧场演绎系统”技术方案。通过统一叙

述框架可以看到，三份文档并不是三条孤立技术线，而是同一个系统在不同层面的展开：软件侧负责把创意和剧本变成结构化资产，Unity 侧负责把结构化资产变成可调度、可训练、可推理的执行任务，机器人侧负责把连续控制量转化为真实动作。

项目的核心创新不只是某个单点算法，也不只是一个会联网的小机器人，而是在于从高层剧本语义到低层连续舵机角度的整条桥接路径被打通。多智能体内容生产让用户能够以创作方式输入任务；统一 JSON 协议保证结构化资产可以被执行层稳定消费；Unity 数字孪生训练和运行时中枢让策略学习与任务调度在同一平台完成；实时舵机角度直控使模型输出能够无缝映射到实体机器人；安全裁决、白名单动作和真实回流又让系统具备现实环境中的可落地性。

因此，本系统已经具备作为比赛答辩平台、具身智能研究原型和教育展示系统的完整雏形。后续只要继续沿着协议统一、训练增强、硬件迭代和日志闭环四条主线持续推进，就能够在保持当前系统完整性的前提下，进一步提升表演性、稳定性和研究价值。

附录 关键接口与示例说明

附录 A 软件到 Unity 的出站说明

软件侧在完成剧本、角色、场景、分镜、图片与音频生成后，将当前集的所有关键资源组装为统一 JSON 并发送到 Unity。本过程不要求 Unity 重新理解自然语言，而是强调结构化字段直达执行层。出站时一般会附带项目标题、角色表、场景表、storyboard 列表、媒体资源路径以及时间线信息。

为保证演示期间资源一致性，建议在单集创建之初锁定图片与音频服务配置；导出前对角色和场景做一次最终校验，确保 ID、名称和资源路径不会在运行期缺失。

附录 B Unity 到机器人控制链路示例

1. Unity 调度器确定当前动作标签为 walk_forward
2. 推理器读取状态向量并输出目标角度：[105, 75, 70, 110]
3. Unity 向机器人发送 /angles
4. 机器人更新四舵机目标值并开始执行
5. 上层等待一小段时间后调用 GET /angles 回读当前角度
6. 若检测到风险或偏差，则限幅、停机或切回白名单动作

附录 C 推荐观测指标

指标	说明	所属层
剧本生成完成率	从创意到标准剧本的流水线完成情况	软件层
角色/场景一致性	同名角色、复用场景及资源绑定是否稳定	软件层
镜头执行完成率	storyboard 节点是否按时完成并正确切换	Unity 层
平均推理时延	每个控制周期内的模型推理耗时	Unity 层
角度到位误差	目标角度与回读角度的差值	机器人层
异常触发次数	越界、碰撞、急停和保底动作切换次数	全链路